

Lösen des Spiels Towers mithilfe von SMT

Philipp Breske

HTWK Leipzig

philipp.breske@stud.htwk-leipzig.de

12. Februar 2025

Zusammenfassung

1 Das Problem

Im Rahmen der Vorlesung *Vorlesung Formale Methoden und Werkzeuge* [3] wurde unter anderem die Methode der SMT Modellierung vorgestellt. Um die Verwendung dieser Methode zu erproben, wurden Puzzle der *Tatham Puzzle Collection* modelliert. [5] Diese Arbeit befasst sich mit dem Spiel Towers.

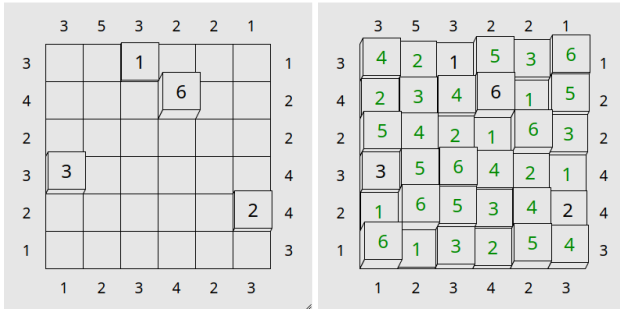


Abbildung 1: Visualisierung des Puzzles

Das Spielfeld b ist eine $n \times n$ Matrix, mit $n \in \{3, \dots, 9\}$. Ziel des Spiels ist es, das gesamte Spielfeld mit Türmen zu besetzen. Dabei hat jeder Turm x eine bestimmte Höhe $h(x)$, wobei $h(x) \in \{1, 2, \dots, n\}$ ist. Weitere Anforderungen an die Lösung sind:

- In jeder Zeile i kommt jede Höhe nur genau einmal vor.

$$\forall j \in \{1, \dots, n\} : \nexists x \in \{1, \dots, n\} \setminus \{j\} : b_{ij} = b_{ix}$$

$$\wedge \forall h \in \{1, \dots, n\} : \exists j \in \{1, \dots, n\} : b_{ij} = h$$

- In jeder Spalte j kommt jede Höhe nur genau einmal vor.

$$\forall i \in \{1, \dots, n\} : \nexists x \in \{1, \dots, n\} \setminus \{i\} : b_{ij} = b_{xj}$$

$$\wedge \forall h \in \{1, \dots, n\} : \exists i \in \{1, \dots, n\} : b_{ij} = h$$

Die Hauptschwierigkeit des Spiels liegt darin, die Anforderungen zu erfüllen, die am Rand des Spielfelds beschrieben sind. Jede Zahl c gibt an, wie viele Türme aus der jeweiligen Richtung sichtbar sind. Zum Beispiel bedeutet die Zahl 1 auf der linken Seite, dass genau ein Turm sichtbar ist, wenn man in dieser Zeile von links nach rechts die Türme betrachtet.

Um dies zu formalisieren, definieren wir die Funktion v , die bestimmt, ob ein Turm sichtbar ist. Die Funktion erhält die Höhe des betrachteten Turms und eine Menge vorausliegender Türme als Eingaben.

$$v(t, ts) = \begin{cases} 1, & \text{falls } \forall x \in ts : t \geq x \\ 0, & \text{sonst} \end{cases}$$

Mithilfe dieser Funktion können nun alle Vorgaben formalisiert werden. Es werden vier Listen co, cu, cl, cr definiert, die die Werte an den Rändern speichern, jeweils für die obere, untere, linke und rechte Seite des Spielfelds.

- Die Anforderungen in co sind eingehalten.

$$\forall j \in \{1, \dots, n\} : \sum_{i=1}^n v(b_{ij}, \{b_{1j}, \dots, b_{ij}\}) = co_j$$

- Die Anforderungen in cu sind eingehalten.

$$\forall j \in \{1, \dots, n\} : \sum_{i=1}^n v(b_{ij}, \{b_{ij}, \dots, b_{nj}\}) = cu_j$$

- Die Anforderungen in cl sind eingehalten.

$$\forall i \in \{1, \dots, n\} : \sum_{j=1}^n v(b_{ij}, \{b_{i1}, \dots, b_{ij}\}) = cl_i$$

- Die Anforderungen in cr sind eingehalten.

$$\forall i \in \{1, \dots, n\} : \sum_{j=1}^n v(b_{ij}, \{b_{ij}, \dots, b_{in}\}) = cr_i$$

Die letzte Einschränkung an die Lösung, ist, dass in Felder schon belegt sein können. Dies wird als Tupel $(i, j, h) \in \{1, \dots, n\}^3$ modelliert. Die schon belegten Felder werden in einer Menge T gespeichert. Für die Lösung gilt dann:

$$\forall (i, j, h) \in T : b_{ij} = h$$

2 SMT

Satisfiability modulo theories (SMT) baut auf klassischen SAT-Solvern auf und erweitert diese um Theorien, wie Arithmetik und komplexere Datentypen, wie Bitvektoren, Integer oder Arrays. Welche Theorien und Logiken verwendet werden können ist auf SMT-LIB [1] standardisiert.

Der Algorithmus, der vielen SMT Solvern zu Grunde liegt, ist DPLL(T). Dieser funktioniert ähnlich wie der DPLL Algorithmus für SAT Solver. Dabei werden alle Atome aus dem SMT-Problem in Booleans übersetzt und darauf dann der klassische DPLL angewendet. Wenn eine erfüllende Belegung gefunden wurde, wird anschließend ein theory solver aufgerufen, der überprüft ob diese Belegung zu einem Theorie-Konflikt führt.

3 Haskell

Haskell ist eine rein-funktionale Programmiersprache, die 1990 entwickelt wurde und nach dem Mathematiker Haskell Curry benannt wurde. Sie zeichnet sich durch immutability (Unveränderbarkeit von Daten) und hohe Abstraktion aus. Haskell legt einen starken Fokus auf mathematische Genauigkeit, Typensicherheit und deklarative Programmierung. Die relevanteste Implementierung stellt der Glasgow Haskell Compiler (GHC) [7] dar, der auch in dieser Arbeit zum Einsatz kommt. Eine weitere Technologie, die im Rahmen dieser Arbeit genutzt wird, ist Cabal [6], welches das Verwalten von Paketabhängigkeiten und das effiziente Bauen und Installieren von Haskell-Projekten unterstützt.

4 Hasmtlib

Hasmtlib ist eine Haskell Bibliothek, die von Julian Bruder entwickelt wurde. [2] Die Bibliothek stellt eine eingebettete domänenspezifische Sprache dar. Eine sogenannte eDSL ist eine Programmiersprache, die speziell für eine bestimmte Problem- oder Anwendungsdomäne entwickelt wurde und in einer Hostsprache eingebettet ist. Dabei nutzt sie die Syntax und Semantik der Hostsprache und erweitert diese,

um bestimmte Aufgaben besonders effizient und ausdrucksstark zu lösen. Die Aufgabe, die Hasmtlib löst, ist die Erstellung von SMT Formeln und die Kommunikation mit den SMT Solvern.

5 Umsetzung

5.1 Lösung

Das Programm besteht aus zwei Teilen. Der eine Teil liest basierend auf der URL das Puzzle ein und extrahiert die wichtigen Daten. Dafür wurde ein Modul *Parser* implementiert, das eine Funktion *getPuzzle* zur Verfügung stellt, welche ein String als Eingabe bekommt und eine Puzzle zurück gibt.

Listing 1: Repräsentation eines Puzzles

```
data Puzzle = Puzzle
{ size      :: Int
, top       :: [Int]
, bottom    :: [Int]
, left      :: [Int]
, right     :: [Int]
, prefilled :: [((Int, Int), Int)]
} deriving Show
```

Das *size*-Attribut gibt die Größe der Matrix an. Die Attribute *top*, *bottom*, *left* und *right* entsprechen den Werten *co*, *cu*, *cl* und *cr* in der Modellierung. Das Attribut *prefilled* enthält eine Liste von Tupeln, die jeweils die Position und die Höhe eines Turms angeben.

```
let puzzle = getPuzzle s
```

Nachdem das Puzzle mit Hilfe des Parsers eingelesen wurde, wird im Programm die Formel konstruiert. Dabei sind einige Constraints recht simpel und einige benötigen Hilfsfunktionen, um lesbar umgesetzt zu werden. Die wichtigste und zugleich komplexeste Anforderung besteht in der Sichtbarkeit der Türme.

```
f (a,b) y =
  ((maxs a y), (ite (y >? a) (b+1) b))

visibleCount x = snd $ foldl f (0, 0) x
```

Für diese Aufgabe wird ein fold implementiert, das in jedem Schritt die Anzahl der sichtbaren Felder sowie die Höhe des höchsten Turms speichert. Die erste Komponente des Tupels repräsentiert die Höhe des bisher höchsten Turms. Dieser Wert wird durch das Maximum des aktuellen Turms und des höchsten Turms berechnet. Die Anzahl der sichtbaren Türme wird erhöht, wenn der aktuelle Turm höher ist als der höchste bisher gesehene Turm. Dabei wird die Formel in jedem Schritt durch *ite* (if-then-else) expandiert.

5.2 Ausführen und Testen

Um das Programm nutzen zu können, müssen *cabal* und der *ghc* installiert sein. Als solver für das System wird Z3 [4] genutzt. Dieser muss ebenfalls auf dem System vorhanden sein.

```
git clone git@gitlab.dit.htwk-leipzig.de:philipp.breske/constraint_project.git
cd constraint_project/
```

Nachdem das Repository geklont wurde, kann das Programm mit *cabal* gebaut und gestartet werden:

```
cabal repl solve
```

Das Programm kann nun interaktiv genutzt werden. Dabei wird die übergebene Formel sowie die berechnete Lösung ausgegeben. Zur Nutzung wird ein Link benötigt, der als Argument für 'main1' in Anführungszeichen übergeben wird:

```
main1 "..."
```

Um die URL zu erhalten, muss nach der Generierung eines Spiels auf der Seite [5] der Link "by game ID" angeklickt werden. Anschließend kann die URL kopiert werden.

Literatur

- [1] Clark Barrett, Aaron Stump und Cesare Tinelli. *The SMT-LIB Standard: Version 2.0*. URL: <https://smt-lib.org/standard.shtml>.
- [2] Julian Bruder. *hasmtlib: A monad for interfacing with external SMT solvers*. 2024. URL: <https://hackage.haskell.org/package/hasmtlib>.
- [3] Johannes Waldmann. *Vorlesung Formale Methoden und Werkzeuge*. 2025. URL: <https://www.imn.htwk-leipzig.de/~waldmann/edu/modul/vorlesung/fmw/>.
- [4] Leonardo de Moura, Nikolaj Bjørner und contributors. *Z3 Theorem Prover*. 2025. URL: <https://github.com/Z3Prover/z3>.
- [5] Simon Tatham. *Simon Tatham's Portable Puzzle Collection*. URL: <https://www.chiark.greenend.org.uk/~sgtatham/puzzles/js/towers.html>.
- [6] The Cabal Team. *Cabal - The Haskell Build System*. URL: <https://www.haskell.org/cabal>.
- [7] The GHC Team. *The Glasgow Haskell Compiler (GHC)*. URL: <https://www.haskell.org/ghc/>.